Cursor Practical Course · Session 1 of 4 · 90 min

Cursor Basics



You'll have AI fix a broken shopping-cart page. Fix the code, and the screen gets fixed. That's the whole 90 minutes.

INTRO

About this course

TODAY

SESSION 1

Basics

Have AI fix a bug, and read the diff it produced.

SESSION 2

Vibe coding

Explain what happens when you ask without deciding anything.

SESSION 3

Spec-driven development

Write the requirements first, then have it build.

SESSION 4

Team work (first half)

Write a spec as a team and open a PR.

SESSION 5

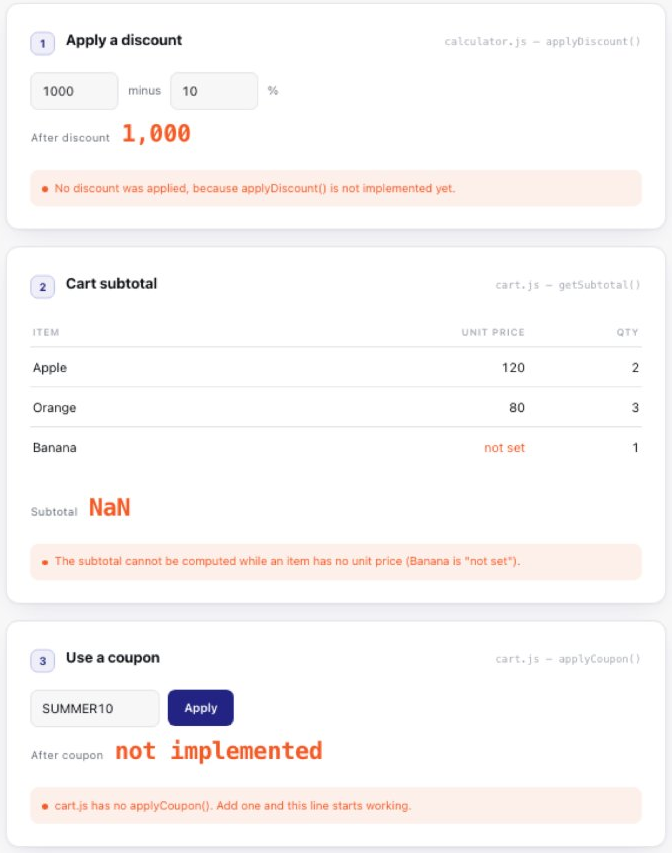
Polish and demo

Get something running and demo it.

Five sessions to "build an app as a team and demo it".

INTRO

What we do today



You'll have AI fix a broken shopping-cart page. Open practice/index.html and three things are broken, in orange.

① Apply a discount

10% off 1000, yet **After discount** still reads **1,000**

② Cart subtotal

Subtotal NaN

③ Use a coupon

not implemented

When all three turn navy, you're done for the day.

INTRO

Three promises

01

You don't have to memorise it

There are only three tools today.

02

You won't be watching for long

75 of the 90 minutes are your own hands on the keyboard.

03

Stuck? Look at your neighbour's screen

Still stuck? Put your hand up.

HOW TODAY RUNS

The 90 minutes

Time	Chapter	What we do	Who
0:00	(Intro)	Cover, the course, what we do today (while downloads run)	You
0:00	1. Getting set up	Install, clone, launch (15 min)	All
0:15	2. Today's goal	The four rungs (5 min)	You
0:20	3. Three tools	Touch Tab / Ctrl+K / Agent one at a time (10 min)	All
0:30	**4. Ask and Agent**	**Ask a question → have Agent implement (15 min)**	All
0:45	**5. Reading the diff**	**Read your own diff, then Keep (10 min)**	All
0:55	6. Hands-on	Do the practice/ exercises yourself (25 min)	All
1:20	7. What you take home	Habits to keep, and what's next (10 min)	You

> **Learners are working for 75 of the 90 minutes. The intro has no slot of its own** — it runs over the download wait in chapter 1.

Cursor Practical Course · Session 1 — Basics

CHAPTER 1 / 0:00 – 0:15

Getting set up

Install Cursor and open the practice page. Nobody moves on until everyone is here.

0:00 — SETUP

Install Cursor, get the files

1. **Download and install it from cursor.com**

   Pick your OS. Mac has separate Apple Silicon and Intel builds

2. **Sign in with a GitHub or Google account**

   Done when the account icon appears, top right

3. **Get the course files (all inside Cursor)**

   Ctrl + Shift + P (Cmd + Shift + P on Mac) → **Git: Clone** → paste the URL below → pick a folder. A terminal works too

```
https://github.com/xrnd-tec/cursor-course.git
```

You don't need to install Node.js today. The practice page runs in the browser alone. Only exercise C (the stretch) uses node, and skipping it is fine.

0:00 — SETUP

Open the page in Cursor's browser

1. **Open the cursor-course/ folder in Cursor**

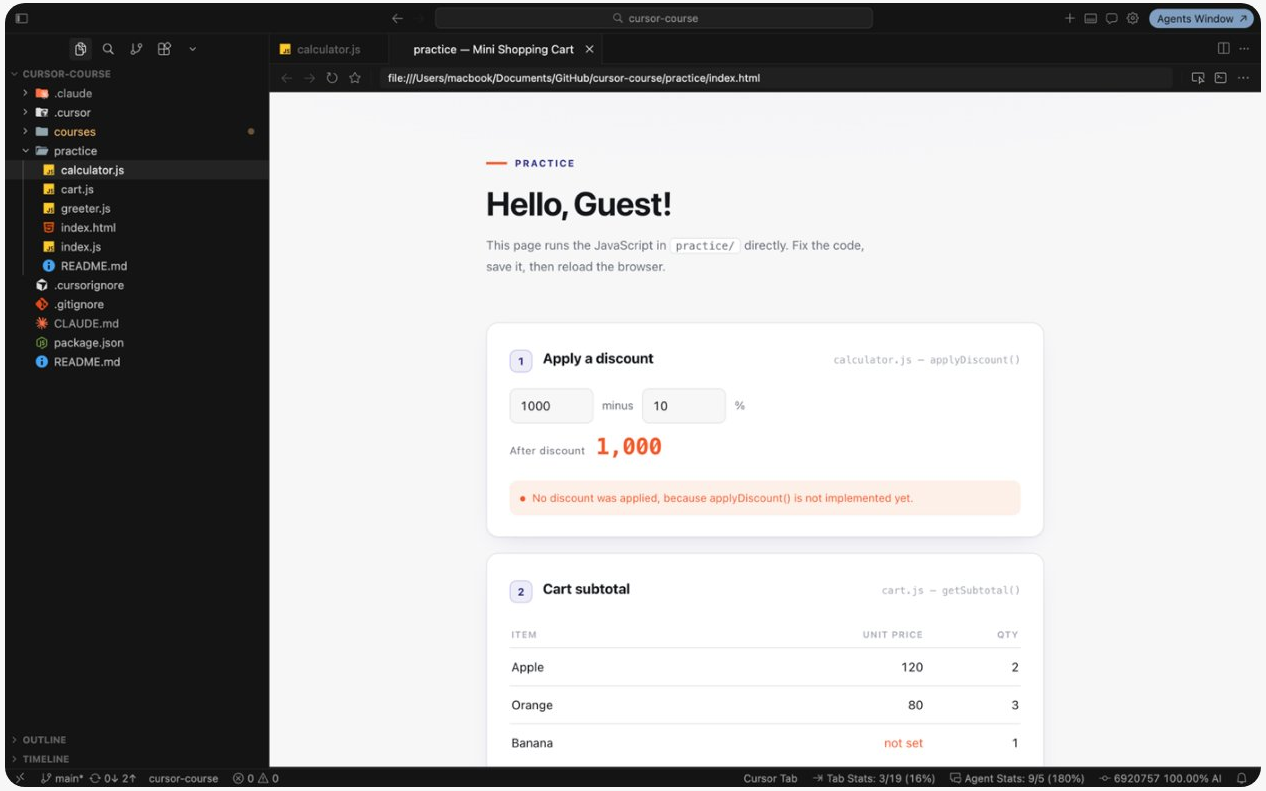
   File → Open Folder

2. **Right-click → Open In Browser**

   On practice/index.html in the sidebar. It opens in Cursor's built-in browser

3. **The "mini shopping cart" page means you're ready**

   ①②③ showing in orange



Open it in your own browser and the numbers stay as "...". ES modules get blocked. Always open it in Cursor's built-in browser.

0:00 — LINE UP FIRST (REQUIRED)

Start Cursor in the IDE view

Once the page opens, **we line up one thing: the screen we use today.**

1. **Cursor Settings → General → Startup → Window Restoration**

   Set it to **Last Used Windows**

2. **Check you're in the IDE view (code in the middle)**

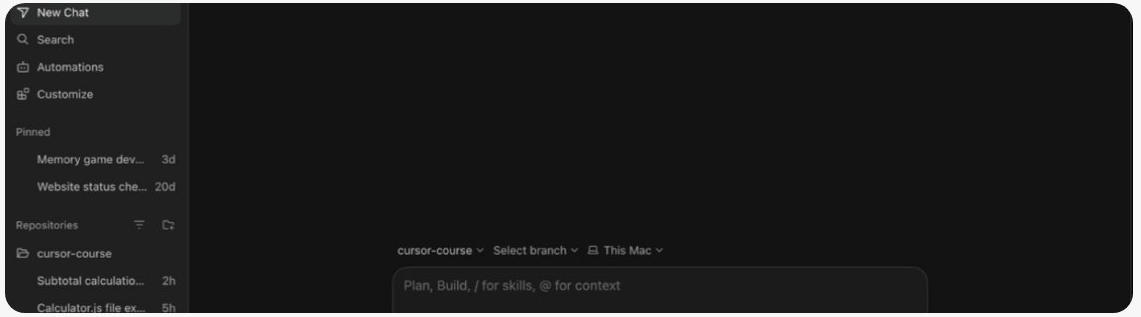
   Ctrl + Shift + P (Cmd + Shift + P on Mac) → Open IDE

3. **Quit Cursor from the IDE so it opens there next time**

   You don't have to quit today

An unfamiliar screen (Agents Window) may come up when you launch. **Nothing is broken.** Today we use the IDE — code in the middle.

On launch (Agents Window)



IDE view (code in the middle)



0:00 — COLUMNS (OPTIONAL)

If you finished early

Both of these are **optional** and not needed today. Do them if you're ahead, or take them home.

COLUMN 1

Set the display language to English

In the IDE, **Ctrl + Shift + P** (Cmd + Shift + P on Mac) → `Configure Display Language` → **English** → full restart.

English is the default, so this is how you get back if Cursor came up in another language. Menus inherited from VS Code follow the setting; **Agent and Cursor Settings are English either way as of August 2026**.

COLUMN 2

Auto-review (for solo work)

Set Run Mode to **Auto-review** under **Settings** → **Agents** → **Execution and Approvals** and safe-looking commands run on their own, while risky ones still ask.

Not required for today's exercises.

0:00 — SETUP CHECK

Get this far first

- [ ] Cursor is running
- [ ] You are signed in (account icon, top right)
- [ ] cursor-course/ is open **in the IDE view**
- [ ] The "mini shopping cart" page is in the built-in browser
- [ ] ①②③ on the page are showing in orange

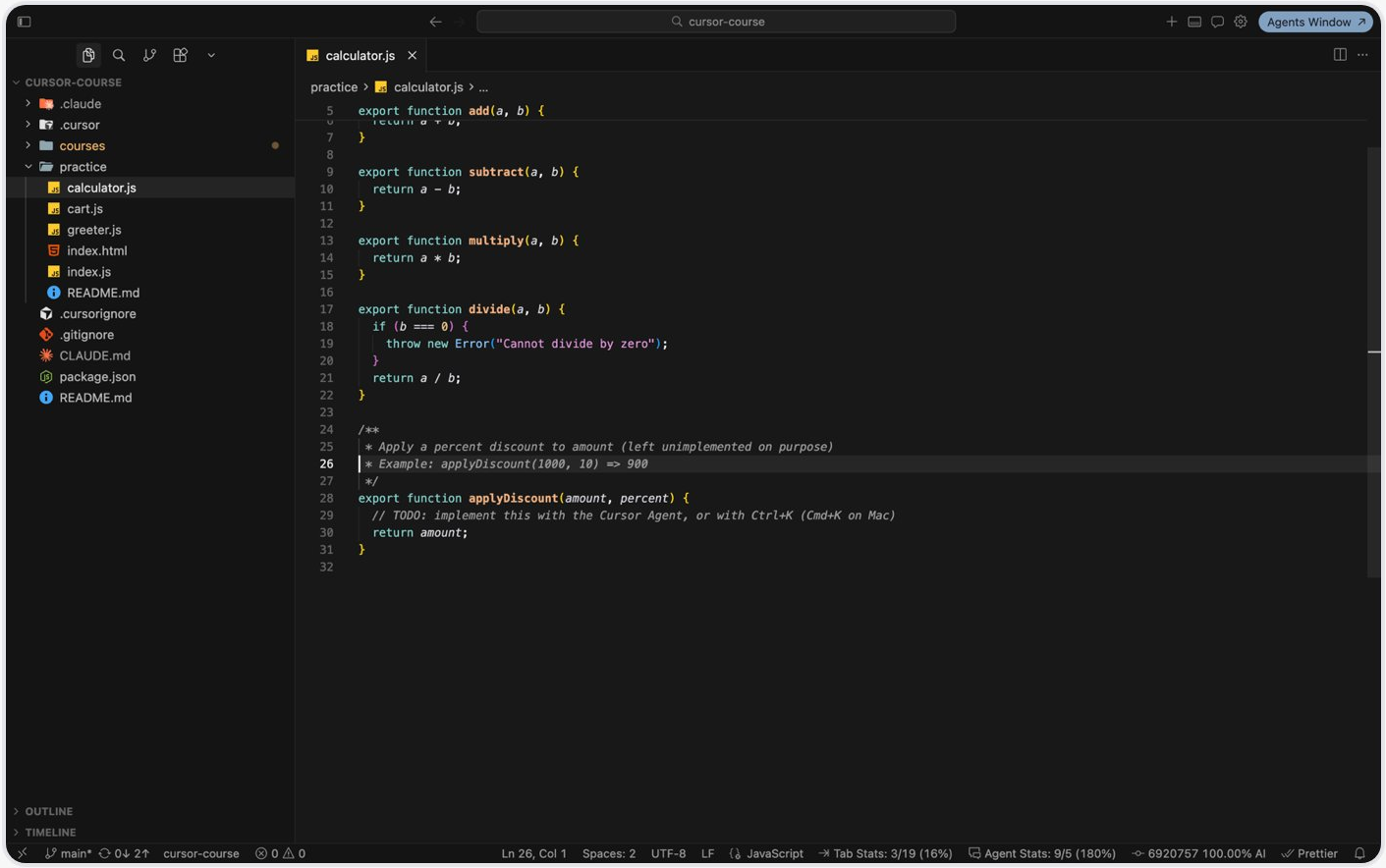
IF YOU'RE STUCK

Slow download	Everyone at once clogs it. Wait
Can't sign in	Use GitHub / Google
Unfamiliar screen	Ctrl+Shift+P → IDE view
Numbers stay "…"	Reopen in the built-in browser
Menus in English	Column 1 changes the language

Nobody moves to chapter 2 until everyone is here. People who finished early don't have to stop.

0:00 — SETUP CHECK

This screen is today's workbench



Sidebar

Opens the files under practice/. Lost it? Ctrl+B (Cmd+B on Mac).

Editor

Only three files today: calculator.js, greeter.js and cart.js.

Agent is still closed

The Agent panel on the right. Ctrl+I (Cmd+I on Mac) opens it. We start using it in chapter 3.

Cursor Practical Course · Session 1 — Basics

CHAPTER 2 / 0:15 – 0:20

Today's goal

Four rungs. We decide how far we climb before we start. This chapter is listening only.

0:15 — TODAY'S GOAL

What you'll be able to do today

01 **Use the basics of Cursor**

Chapter 3 — the greeting in the heading changes

02 **Have AI explain existing code**

Chapter 4 (ask with @ in Ask mode)

03 **Have AI fix a bug**

Chapter 6, exercise A — **② the subtotal appears**

04 **Have AI build something new**

Chapter 4 / exercise B — **① the discount works / ③ the coupon works**

STRETCH

You implement applyDiscount in chapter 4, so everyone passes through the door to 04 once. 03 is the line for everybody.

Cursor Practical Course · Session 1 — Basics

CHAPTER 3 / 0:20 – 0:30

Three tools

Tab, Ctrl+K, Agent. You touch all three yourself, one at a time.

0:20 — CHAPTER 3

Three ways to ask AI

`Tab`

Suggests what's next

Grey text appears as you type. Tab takes it, Esc drops it. It reads the surrounding code and your recent edits.

`Ctrl + K`

Edits just this

Rewrites only what you selected: "make this async", "change this wording". Cmd + K on Mac.

`Ctrl + I`

Changes things by talking

The Agent panel on the right. Reads several files, answers, edits. Today's main tool. Cmd + I on Mac.

> **Shortcuts in this course are written for Windows.** On a Mac, read the brackets.
> Today's screen: **the IDE view (code in the middle) + the Agent panel on the right**. We don't open Agents Window today.

0:20 — CHAPTER 3

The keys you'll press

What you want	From the screen	Windows	Mac
Show / hide the sidebar	Icon, top left	**Ctrl + B**	Cmd + B
Open the Agent input	Agent panel on the right	**Ctrl + I**	Cmd + I
Change the mode	Click the mode name, bottom left of the input	**Shift + Tab (cycles)**	Shift + Tab
Change the model	Click the model name by the input	**Ctrl + /**	Cmd + /
Edit only the selection	(shortcut only)	**Ctrl + K**	Cmd + K
Take the suggestion	(shortcut only)	**Tab**	Tab
Get back to the IDE view	Command palette → Open IDE	**Ctrl + Shift + P**	Cmd + Shift + P

> Shortcuts can be remapped. **If one doesn't fire, use the button on screen instead.** Getting stuck here costs you the rest.

0:20 — CHAPTER 3

What Tab is good at, and what it isn't

What Tab reads: the surrounding code, your recent edits, linter output. That's why it's strong at continuing the line you're on.

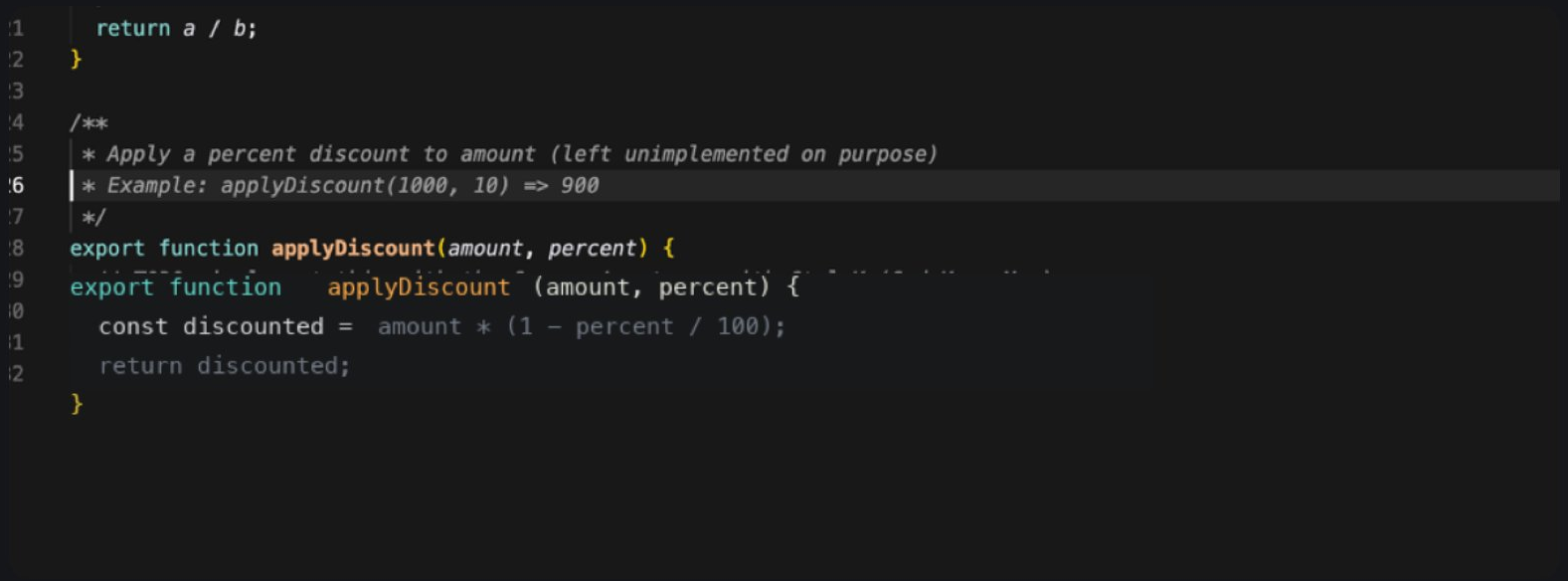
Tab shines	Don't leave it to Tab
Repeating a similar pattern	Logic that needs a product decision
Imports and type completion	Security or money calculations
Continuing what you're writing	Anything that must be reviewed

> More: courses/fundamentals/00-map.md · 04-tab-and-inline.md

0:20 — CHAPTER 3 ⏲ (3 MIN)

First, make Tab show up

Open **practice/calculator.js**, put the cursor inside applyDiscount and start typing something.



> Grey means "not in the file yet". **Tab** takes it, **Esc** drops it. Nothing changes until you take it.

0:20 — CHAPTER 3 ② (5 MIN)

Ctrl + K edits one spot

Open **practice/greeter.js**, select the whole greet function and press **Ctrl + K** (Cmd + K on Mac). Paste this and send.

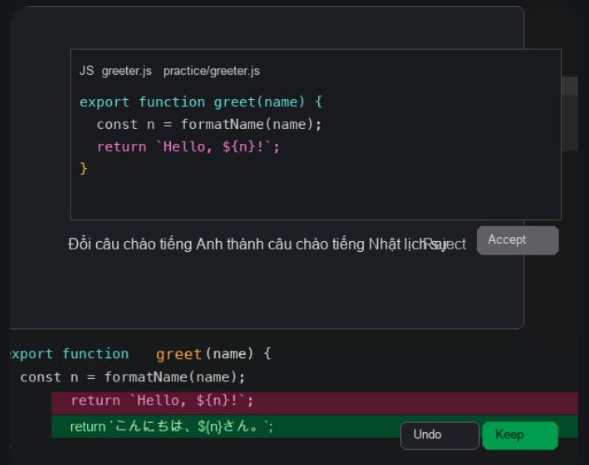
```
Change the English greeting to a polite
Japanese greeting.
```

> When it lands, choose **Accept** or **Reject**. This is the one place labelled Accept / Reject instead of Keep / Undo.

① Select, then Ctrl + K



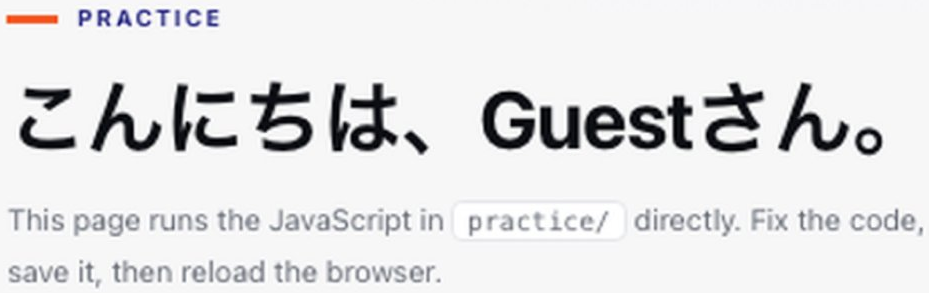
② It becomes a diff, in place



0:20 — CHAPTER 3 ②

Accept it, then reload the page

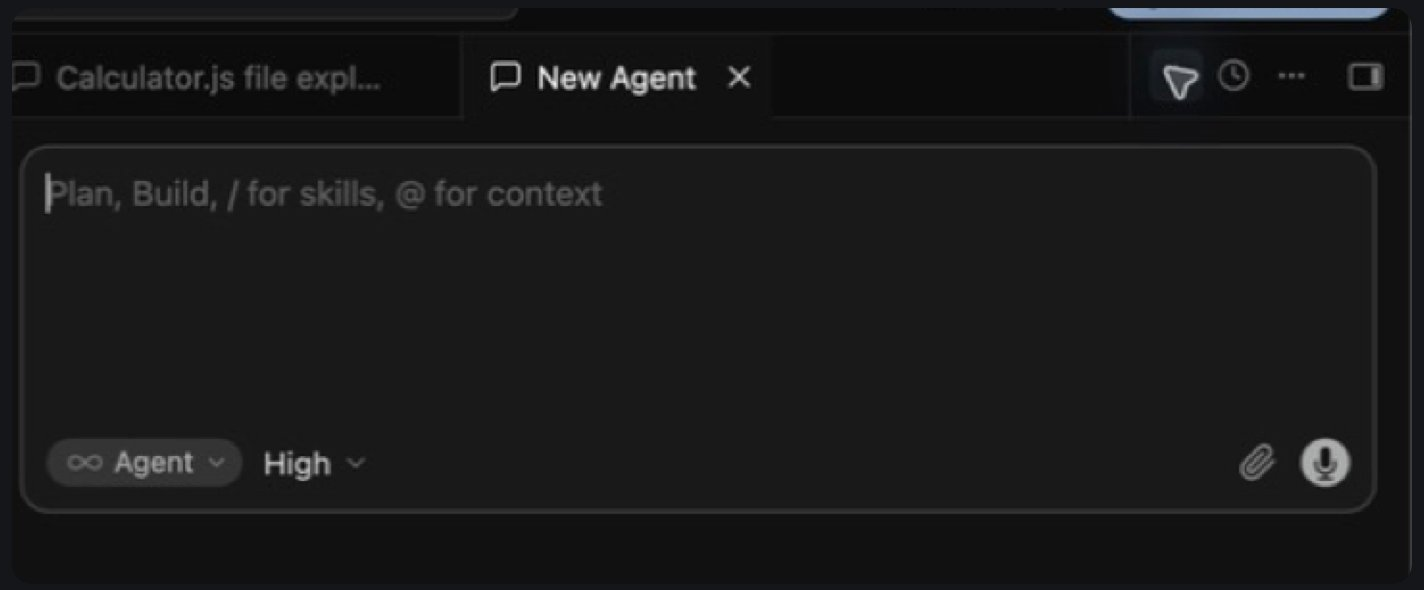
Reload the browser and **the heading on the page changes.**



This is the first time today that **your own change shows up on screen**. "What you edited was code, but what changed was the screen."

0:20 — CHAPTER 3 ③ (2 MIN)

Just open the Agent input



> **Ctrl + I** (Cmd + I on Mac) opens the Agent panel on the right. We're not using Agents Window today. **Send nothing yet** — just open it.

0:20 — CHAPTER 3

Which one, when

I want the rest of this line	**`Tab`**
Just fix what I selected	**`Ctrl + K`**
I want to understand — don't write yet	**`Ask`**
Build it, fix it, run it	**`Agent`**
Big change — plan it first	**`Plan`**

> Tab is "what's next", Ctrl + K is "just here", Agent is "by talking". **Today we mostly use Agent.**

Cursor Practical Course · Session 1 — Basics

CHAPTER 4 ／ 0:30 – 0:45

Ask and Agent

The heart of today. Understand it with Ask, then let Agent do the editing.

0:30 — CHAPTER 4

The mode decides how AI behaves

Mode	Good for	Edits files
Agent	Features, refactors, bug fixes, running tests	Yes
Plan	Agreeing the approach before a big change	After you approve
Debug	Chasing a bug that's hard to reproduce, with evidence	Yes
Multitask	Running independent jobs in parallel	Yes (in parallel)
Ask	Questions, explanations, design talk (nothing written yet)	**No (read-only)**

> It sits at the bottom left of the input. Click to pick one, or cycle with **Shift + Tab**. **Today you only need Ask and Agent.**

0:30 — CHAPTER 4

Mode and model are different things

MODE

How AI behaves

Whether it edits, or plans first.

Bottom left of the input. Cycle with **Shift + Tab**.

MODEL

The brain handling it

Which AI answers you.

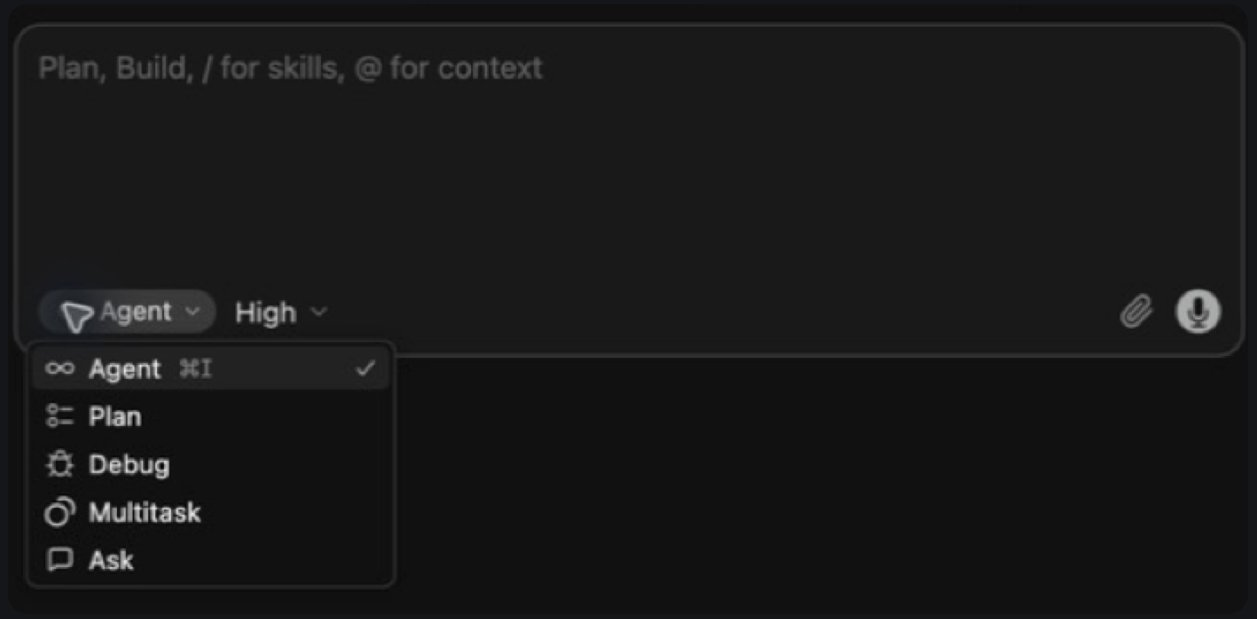
Next to the input. **Ctrl + /** (Cmd + / on Mac).

> The default **Auto** doesn't mean "no model" — it means **Cursor Router picks one per request**. Harder tasks go to stronger models, simple ones to cheaper models.

Leave it alone today.

0:30 — CHAPTER 4 (2 MIN)

Click the mode name and pick Ask



Click the mode name at the **bottom left** of the input and this list opens.

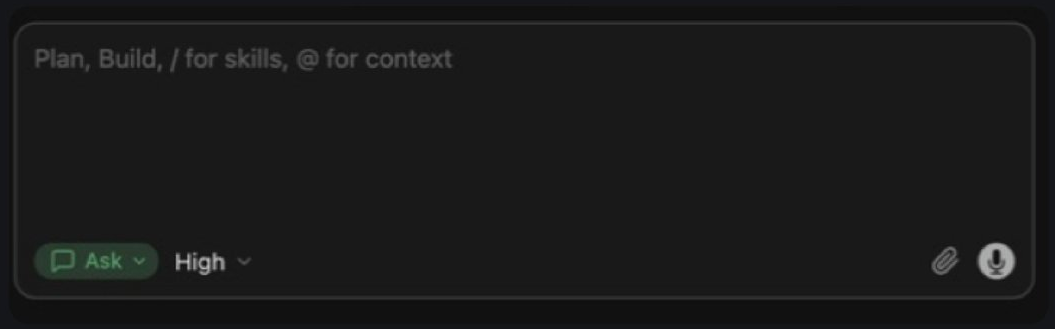
Today you only need **Ask** and **Agent**.

Shift + Tab changes it too, but it cycles without opening the list.

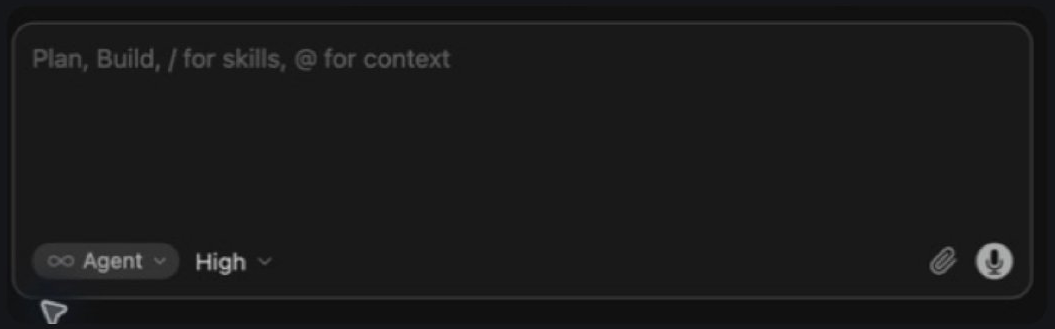
0:30 — CHAPTER 4

This is where you check it changed

Now on Ask



Back on Agent



> The bottom left of the input is your current mode. **Check it with your own eyes before every send.** Asking Agent-work in Ask mode is the first place people trip.

0:30 — CHAPTER 4

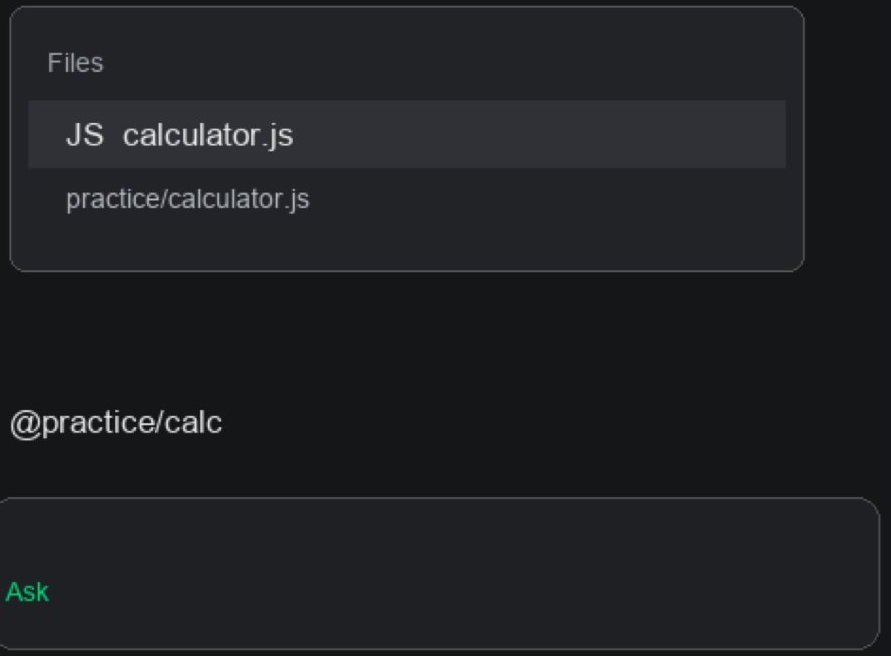
@ points at what to put in the conversation

What you type	When to use it
@filename	"Answer / edit with this file in mind"
@folder/	"Only look in here"
@Docs	Reference documentation you've registered
@Terminals	Show it error output or run results
@Past Chats	Pick up an earlier conversation
@Commit / @Branch	Uncommitted changes, or the diff against main

> The rule of thumb: **@ it when you know the file. When you don't, leave it out and let Agent search.** Dumping a big unrelated folder in is just noise. **One file is plenty today.**

0:30 — CHAPTER 4

Type @ and the list appears



> Type @ and suggestions come up. Pick one — otherwise Cursor answers without reading the file.

0:30 — CHAPTER 4

Same request, different result

WEAK REQUEST

```
fix the bug
```

GOOD REQUEST

```
@practice/cart.js @Terminals
Fix the "subtotal comes out NaN" problem.
Repro: addItem with an item that has no
price, then call getSubtotal.
```

> Target, symptom, how to reproduce. Those three change what comes back.

0:30 — CHAPTER 4 ② (4 MIN)

Ask a question first

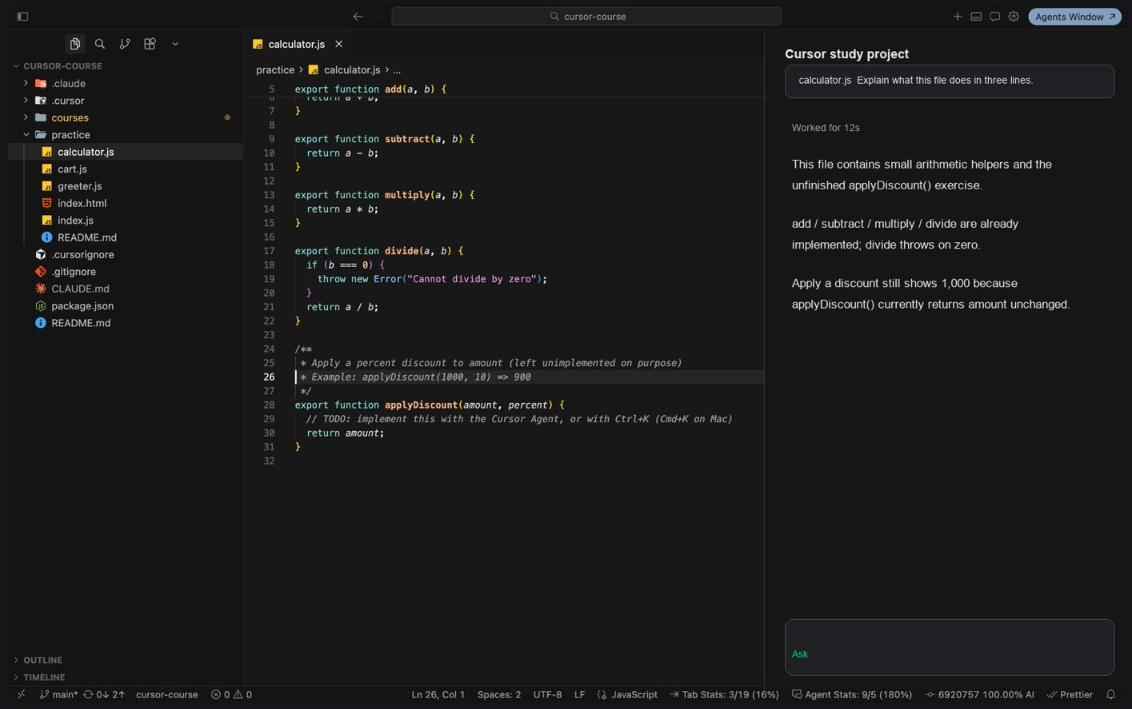
1. Ctrl + I (Cmd + I on Mac) opens the Agent panel on the right
2. Click the mode name at the bottom left and pick **Ask**
3. Send the prompt below

```
@practice/calculator.js
Explain what this file does in three lines.
Then tell me why "Apply a discount" on the page has no effect.
```

> Nothing comes back for 30–60 seconds. @ takes folders and error output too, not just files — though one file is plenty today.

0:30 — CHAPTER 4 ②

Ask only answers



You get an answer, but **nothing changed in the editor**.

No diff either.

Ask when you want to understand. Agent when you want a change.

0:30 — CHAPTER 4 ③④ (9 MIN)

Switch back to Agent and have it build

Click the mode name and go back to **Agent** before you send.

```
@practice/calculator.js
Implement applyDiscount(amount, percent).
Example: applyDiscount(1000, 10) → 900
You don't need to keep the TODO comment
```

Name the target

Point at the file with @

Give an example

One input and the output you expect

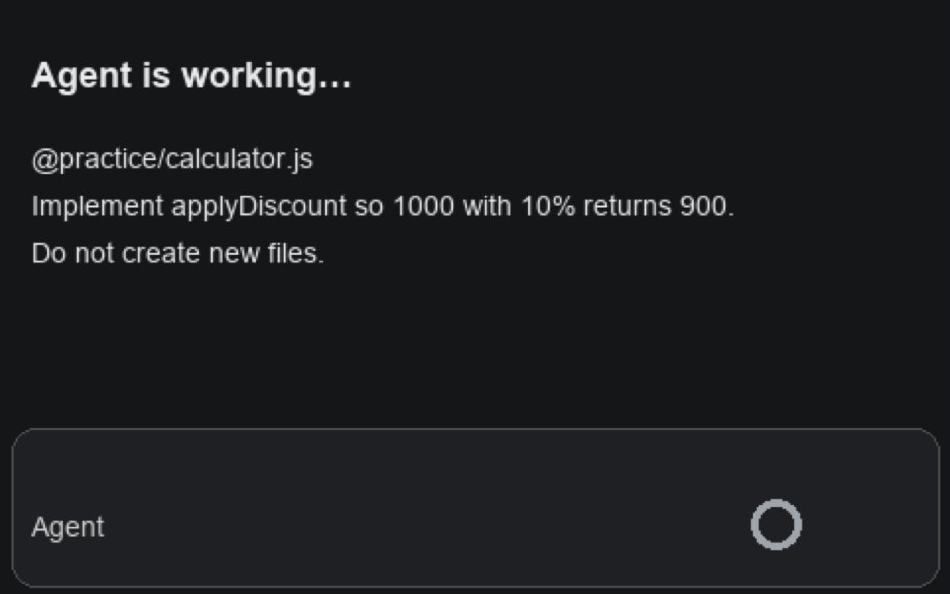
State when it's done

One line: what makes it finished

Wait for the diff. Don't Keep it yet. We read it in the next chapter and decide then.

0:30 — CHAPTER 4

Send it, then wait



This is what thinking looks like.

It can take 30–60 seconds.

To stop it, hit the stop button right of the input.

Cursor Practical Course · Session 1 — Basics

CHAPTER 5 / 0:45 – 0:55

Reading the diff

Read the diff you produced, then Keep it. The most important chapter today.

0:45 — CHAPTER 5

The change is already in the file

It isn't "review, then apply". Agent writes to the file as it works. That's exactly why you read before you decide.

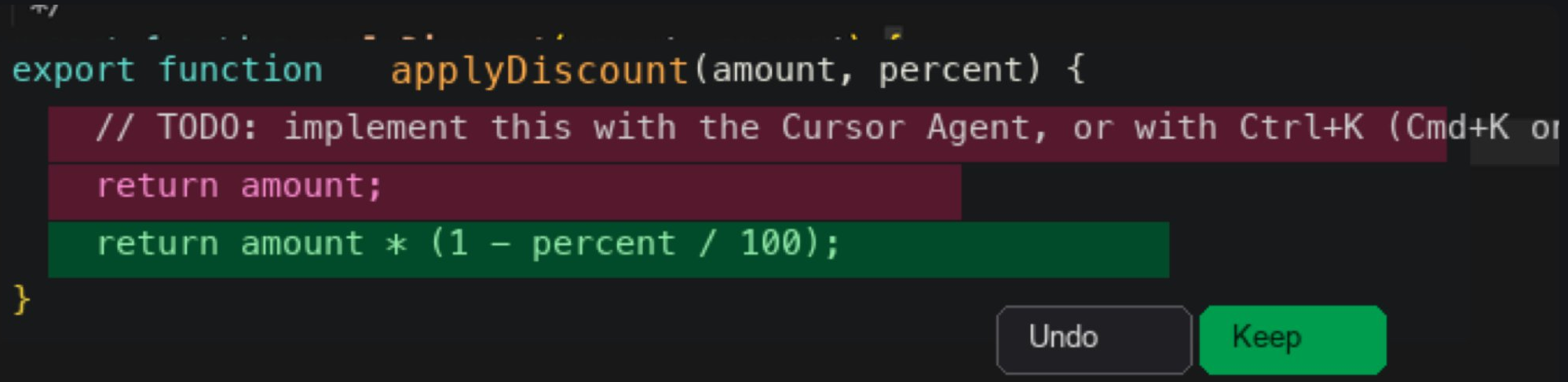
Button	What it does	Where it is
Keep	Take the change	Bottom right of the diff, or the change bar in the Agent panel
Undo	Put it back	Same places
Review	Go through every change at once	Bottom of the Agent panel
Restore Checkpoint	Rewind all the way to that point	Next to the message

> **Green** is an added line, **red** is a removed one. Even when you're sure you took everything, build the habit of a final check with git or your own eyes.

0:45 — CHAPTER 5 (4 MIN)

This is a diff

Red goes away, green comes in. Read **the diff you produced**, line by line.



> **Undo** / **Keep** at the bottom decide whether this diff stays. **Read the green and red before you press.**

0:45 — CHAPTER 5

Three questions while you read

01

Did anything change that I didn't ask for?

Extra files, extra lines slipping in.

02

Are the removed lines (red) safe to lose?

Read the red while you can still get it back.

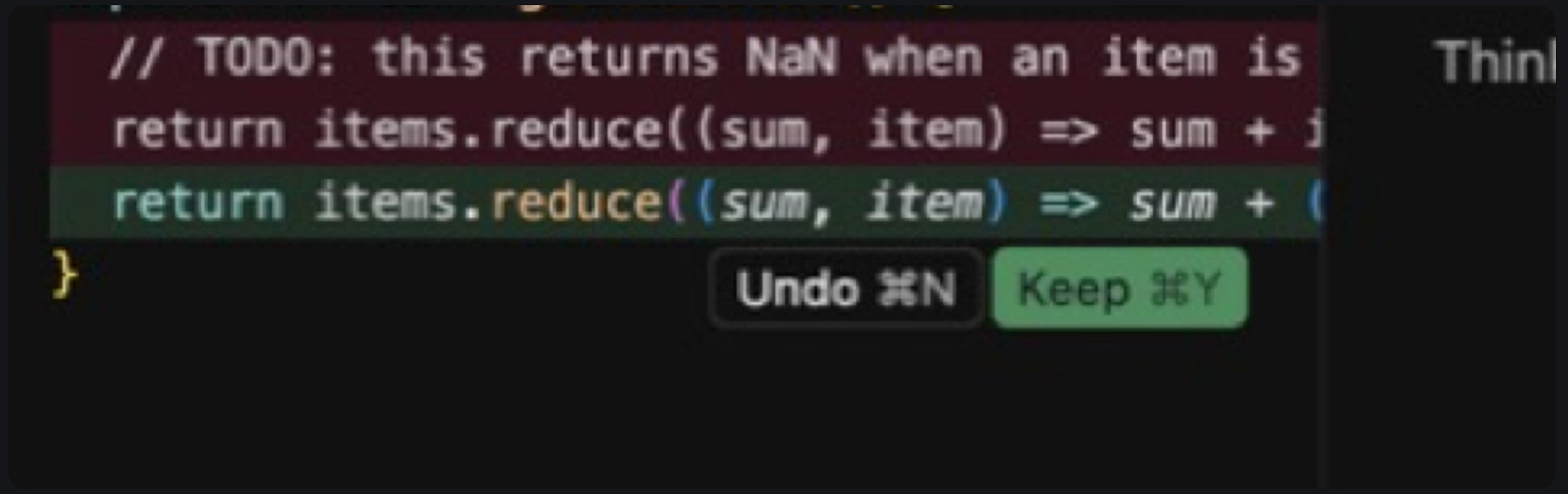
03

Will applyDiscount(1000, 10) return 900?

Read the code and judge it yourself.

0:45 — CHAPTER 5 ② (2 MIN)

You choose: Keep or Undo



> Happy with it? **Keep**. Not happy? **Undo** and ask again. To rewind further, use **Restore Checkpoint** next to the message.

0:45 — CHAPTER 5 ②

Keep, and the file changes

```
export function applyDiscount(amount, percent) {
  return amount * (1 - percent / 100);
}
```

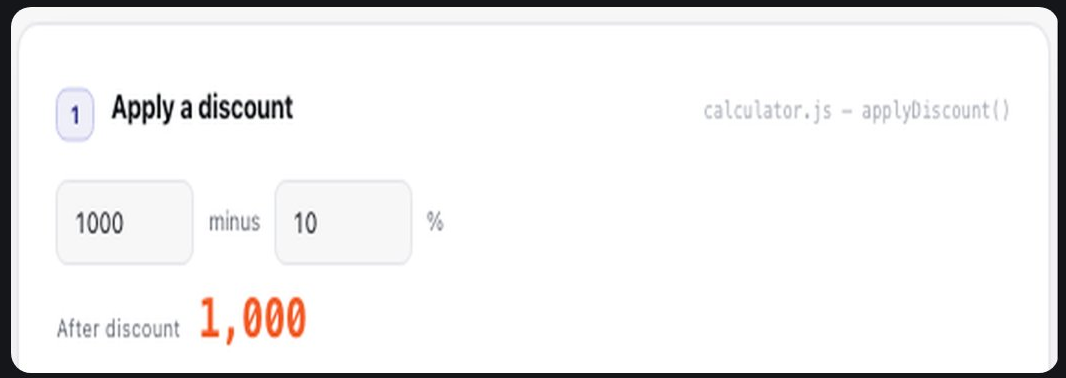
> The colors are gone and applyDiscount is implemented. **But we still haven't looked at the screen.**

0:45 — CHAPTER 5 ③ (2 MIN)

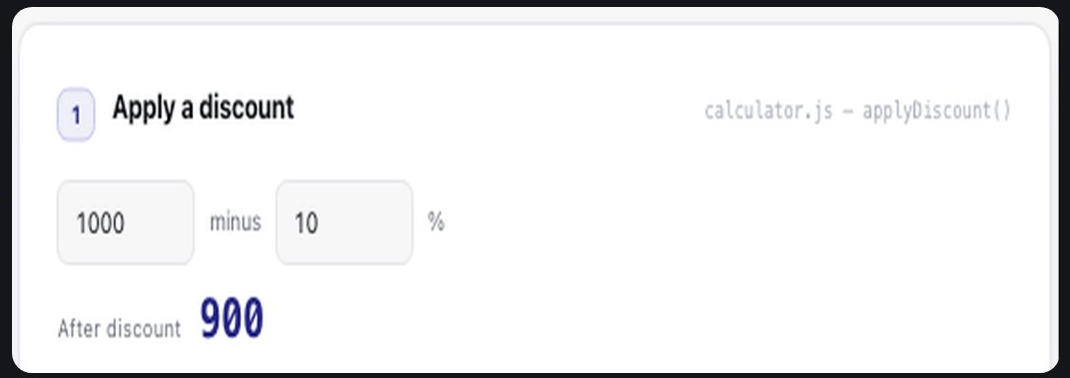
Check it on the screen

Reload the browser.

Until just now



After the reload



You fixed the code, and the screen got fixed. That's one full cycle. If the number didn't move, either Keep didn't land or the implementation is wrong.

0:45 — CHAPTER 5 ⏱ (2 MIN)

Check your work with Ask

Switch the mode to **Ask** and send this.

```
@practice/calculator.js
What should applyDiscount(1000, 10) return? Is the current code right?
```

> Ask again after the fix. That's part of the cycle too.

TODAY'S LOOP

Always in this order

01

Ask

`@file + question`

Understand it first

02

Agent

`@file + request + done`

Have it changed

03

diff

`green and red, line by line`

Check it's what you asked

04

Keep / Undo

`take it, or put it back`

You decide

> **Always read the diff before you Keep.** That's the ground floor of using Cursor safely.

Cursor Practical Course · Session 1 — Basics

CHAPTER 6 ／ 0:55 – 1:20

Hands-on

The instructor stops talking here. 25 minutes, everyone writing.

0:55 — CHAPTER 6

Requests have a shape

Before you worry about the model, write the goal, the limits and what "done" means. Every exercise prompt today has this shape.

```
[Goal] one line
[Target] @file or folder (as far as you know)
[Limits] what not to break / what you may use
[Done when] what has to be true
[Not needed] (optional)
```

Avoid	Say instead
make it nice	the look, the behaviour, the edge cases
refactor everything	the files, and what's in and out of scope
fix the bug	expected, actual, how to reproduce
optimise it	speed, readability or compatibility — which wins

> **New task, new chat.** Old assumptions get in the way less often.

0:55 — EXERCISE A (EVERYONE)

Fix the subtotal in cart.js

Goal 03 — screen ②. Subtotal reads **NaN**. Same order as chapter 4: check with Ask, then hand it to Agent.

① Ask

```
@practice/cart.js
Tell me one or two problems with getSubtotal.
Just checking before I change anything.
```

Read the diff, **Keep** it, then **reload the browser**.

② Agent

```
@practice/cart.js
Improve getSubtotal with these rules.
- skip items where price or qty is
  undefined / null
- keep returning 0 for an empty cart
- don't rename the function or its arguments
```

Until just now

② Subtotal **NaN** (orange)

After the reload

② Subtotal **480** (navy)

0:55 — EXERCISE A

This is what done looks like

```
export function getSubtotal() {
  // TODO: this returns NaN when an item is missing price or qty. Make it survive that
  return items.reduce((sum, item) => sum + item.price * item.qty, 0);
  return items.reduce((sum, item) => {
    if (item.price == null || item.qty == null) {
      return sum;
    }
    return sum + item.price * item.qty;
  }, 0);
}
```

Undo Keep

> If items with a missing price or qty get skipped, you're done. Your code doesn't have to match this.

0:55 — EXERCISE B (IF YOU HAVE TIME)

Implement the coupon

Goal 04 — screen ③. The stretch. Start it once exercise A is done.

```
@practice/cart.js @practice/calculator.js
Add applyCoupon(code) to cart.js.
- for SUMMER10, return getSubtotal minus 10%
- for an invalid code, return getSubtotal as is
- you may use applyDiscount from calculator.js
- don't create new files
```

Keep it, reload, and **not implemented** on screen ③ turns into an amount. **That's all three spots navy.**

※ It won't work without applyDiscount. If you hit Undo in chapter 4, implement that first. The function must be called **applyCoupon** and it needs **export**.

0:55 — EXERCISE C (IF YOU STILL HAVE TIME)

Learn the fix-from-an-error loop

Same logic, this time **in the terminal**. **practice/index.js** is still an empty placeholder.

```
@practice/index.js @practice/cart.js
Write code in index.js that console.logs the cart contents and the subtotal.
Done when: node practice/index.js runs without errors and prints the amount
```

Run it in the terminal

```
node practice/index.js
```

If you get an error, paste the terminal output straight into Agent and have it fixed. That's the loop for errors.

※ Skip this one if you don't have node installed.

Cursor Practical Course · Session 1 — Basics

CHAPTER 7 / 1:20 – 1:30

What you take home

The three habits, the hand-in, and what comes next.

1:20 — CHAPTER 7

What to take home

01

Check with Ask before you hand it to Agent

Don't change things blind. Understand first.

02

Name the target with @

A vague request gets you a vague result.

03

Read the diff before you Keep

Don't over-trust Cursor. You're the one deciding.

1:20 — HAND-IN (30 SECONDS)

One screenshot of the page

One screenshot of the cart page,
into the chat or the shared folder.

One orange turning navy is enough. It doesn't have to be all three.

WHAT WE LOOK AT

How many oranges are left tells us at a glance who got how far. **It decides how session 2 opens.**

1:20 — CHAPTER 7

What's protected by default

Today's repo is for learning, so nothing you do here matters. At work, be careful not to let Agent read API keys or customer data.

Area	Default behaviour
Reading and searching files	No approval needed
Writing to files	Saved immediately — which is why version control comes first
Running terminal commands	**Follows Run Mode** (Auto-review if you're working solo)
Network	No arbitrary traffic. GitHub, links you name, web search and the like only

> Keep files out with **.cursorignore**. Better still, **don't put secrets in the repo at all**. Working solo, Auto-review is worth it (column 2 in chapter 1).

NEXT TIME — SESSION 2

Next: you build a memory game

You ask for it to be made "nicely" without deciding anything first, and at the end everyone shows their game.

REVIEW (ON YOUR OWN)

Look up only what tripped you

`00-map.md`	**The whole picture**	Tab / Ctrl+K / Agent, and the first settings
`01-modes.md`	**Modes in detail**	Agent / Ask / Plan / Debug and Multitask
`03-context.md`	**Every kind of @**	Files, folders, error output
`05-prompting.md`	**Asking well**	Turning a vague request into one that can be run

`all under courses/fundamentals/`